Informe Técnico: Migración Arquitectónica de FastAPI a Inferencia Nativa en el Navegador con ONNX

Proyecto: SportMind Dashboard

Objetivo: Documentar el proceso de desacoplamiento del backend en la nube (FastAPI) y la migración hacia una arquitectura 100% autónoma en el frontend mediante la conversión de modelos de Machine Learning a ONNX y su ejecución local en JavaScript.

1. Contexto y Diagnóstico Inicial

Durante el desarrollo del sistema de predicción de autorregulación emocional, el proyecto operaba bajo una arquitectura tradicional de cliente-servidor:

- **Dependencia externa:** El frontend en Astro realizaba peticiones `fetch` vía HTTP a una API desplegada en Render construida con **FastAPI** y **Uvicorn**.
- **Desafíos de integración:** Se presentaron fricciones operativas comunes en entornos de desarrollo web, tales como restricciones de políticas CORS, manejo de rutas preliminares (*preflight options*) y la latencia derivada de los arranques en frío (*cold starts*) de los servicios en la nube.
- **Propósito de la migración:** Con el fin de lograr una aplicación completamente independiente, autónoma y de respuesta instantánea, se evaluó y ejecutó la transición hacia un modelo de inferencia local integrado directamente en el navegador web.

2. Eliminación del Servicio Intermedio (FastAPI)

Para simplificar la infraestructura y reducir costos de mantenimiento operativo:

- Se removió la necesidad de mantener un servidor en Python en paralelo ejecutándose de manera constante para procesar las peticiones del modelo.
- Se eliminaron las llamadas remotas `POST` desde el dashboard hacia endpoints externos, trasladando todo el procesamiento matemático y lógico al cliente (navegador del usuario).

3. Conversión del Modelo (.pkl a .onnx)

Dado que los navegadores web interpretan nativamente JavaScript y no soportan objetos binarios o entornos de Python (`scikit-learn` / `numpy`), se utilizó el estándar abierto **ONNX (Open Neural Network Exchange)** para exportar el modelo entrenado:

- **Desactivación de ZipMap:** Durante el proceso de conversión en Python utilizando `skl2onnx`, se configuró explícitamente la opción `{'zipmap': False}` para evitar que las probabilidades del modelo se exportaran como diccionarios complejos incompatibles con entornos web.

- **Generación del artefacto:** Se transformó el archivo `modelo_autoregulacion_emocional.pkl` en un archivo binario optimizado llamado `modelo_autoregulacion.onnx`, el cual fue alojado como recurso estático dentro de la carpeta `public/` del proyecto Astro.

4. Implementación Nativa en el Frontend (JavaScript / ONNX Runtime)

La integración final se completó incorporando la librería `onnxruntime-web` directamente en el componente del dashboard:

- **Mapeo y Codificación local:** Se replicaron funciones auxiliares en JavaScript (`codificarDeporte`, `codificarGenero`, `codificarEmocion`) para simular el comportamiento de los `LabelEncoders` originales de Python, transformando las variables categóricas en los índices numéricos que espera la red neuronal.

- **Inferencia Dinámica:** Mediante la lectura automatizada de las salidas de la sesión (`session.outputNames`), el sistema ejecuta el modelo localmente en la memoria RAM del dispositivo del usuario:

    - Procesa un vector bidimensional de 20 características.
    - Extrae con precisión la etiqueta de predicción (`output_label`).
    - Obtiene de forma segura el arreglo de probabilidades (`output_probability`) para calcular y mostrar el porcentaje de certeza del modelo de manera instantánea.

5. Conclusión de la Arquitectura Actual

Con esta implementación, el dashboard de SportMind pasó a ser un sistema robusto, privado y autónomo:

- **Ventajas obtenidas:** Eliminación total de latencia en red, operaciones sin dependencias de servidores externos y ejecución inmediata al hacer clic en los botones de análisis de las sesiones de los atletas.